

1(a) * যখন subclass ও superclass একই package এর মধ্যে থাকবে
যেহেতু subclass, superclass এর protected member কে
access করা হবে

⇒ when the subclass and superclass are in the same
package, the subclass accesses protected member
just like public ones or directly. It can access them
by creating an instance of the superclass.

Example:

```
package p1;  
class Parent {  
    protected int x = 10;  
}
```

```
class Child extends Parent {  
    void show() {  
        System.out.println(x); // Allowed  
    }  
}
```

?

Here, child can access x directly.

* যখন subclass ও superclass different class package
located

⇒ when the subclass is in a different package, it
can only access the protected member through inheritance.
It cannot access the member using an instance of the
Superclass.

Example:

```
// package A
```

```
package P1
```

```
public class Parent {
```

```
    protected void display() {
```

```
        System.out.println("Hello from Parent");
```

```
    }
```

```
// package B, extends P1, extends P2
```

```
package P2
```

```
import P1.Parent;
```

```
public class Child extends Parent {
```

```
    public void test() {
```

```
        display(); // work Accessed via inheritance
```

```
    }
```

```
public class Main {
```

```
    public void test() {
```

```
        Parent p = new Parent();
```

```
        // P.display(): // Error: display() has protected access
```

```
        in Parent
```

```
    }
```

- Access is allowed only through inheritance.

3. How does encapsulation ensure data security and integrity?
 ? show with a BankAccount class using private variables and validate methods such as setAccountNumber(String), setInitialBalance(double) that rejects null, negative and empty values.

= Encapsulation a core principle of object oriented programming, enhances data security and integrity by bundling data and methods that operate on that data within a single unit and restrict direct access to the data.

It protects data from -

- i) Direct unauthorized access
- ii) Invalid or harmful input
- iii) Inconsistent object states

Work process of encapsulation is given below with a BankAccount class:

Code:

```
public class BankAccount {
    private String accountNumber;
    private double balance;

    public void setAccountNumber(String accountNumber) {
        if (accountNumber == null || accountNumber.trim().isEmpty()) {
            System.out.println("Invalid number.");
        }
    }
}
```

```
else {
    this.accountNumber = accountNumber;
}
}

public void setInitialBalance(double balance) {
    if (balance < 0)
        System.out.println("Invalid balance.");
    else
        this.balance = balance;
}

public String getAccountNumber() {
    return accountNumber;
}

public double getBalance() {
    return balance;
}

public void deposit(double amount) {
    if (amount < 0) {
        System.out.println("Invalid amount.");
    }
    else {
        balance += amount;
        System.out.println("Deposited: " + amount);
    }
}
}
```



```

public class BankApp {
    public static void main (String[] args) {
        BankAccount acc = new BankAccount ();
        acc.setAccountNumber(" "); # invalid
        acc.setAccountNumber("ACC2304"); # valid
        acc.setInitialBalance(-1000); # invalid
        acc.setInitialBalance(1000); # valid
        acc.deposit(-500); # invalid
        acc.deposit(500); # valid

        System.out.println("Final Balance: " + acc.getBalance());
    }
}

```

So the benefit of using encapsulation here is:

- i) private variables → prevent direct external manipulation.
- ii) validate setters → Rejects bad input such as: null, empty, negative etc.
- iii) controlled methods → Ensures proper business logic (deposit).

4. Write Program to (Any 3) →

i) Find the kth smallest element in an ArrayList

Code:

```

import java.util.*;

public class KthSmallestElement {
    public class int KthSmallest(ArrayList<Integer> list, int k) {
        Collections.sort(list);
        return list.get(k-1);
    }

    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        ArrayList<Integer> list = new ArrayList<>();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++)
            list.add(sc.nextInt());

        int k = sc.nextInt();
        int smallest = findKthSmallest(list, k);
        System.out.println("The " + k + "th smallest element is: " + smallest);
        sc.close();
    }
}

```


ii) Create a TreeMap to store the mapping of words to their frequencies in a given text.

Code:

```
import java.util.*;
public class WordFrequencyTreeMap {
    public static void main (String[] args) {
        String text = "I am Maisha";
        String[] words = text.split(" ");
        TreeMap<String, Integer> map = new TreeMap<>();
        for (String word : words) {
            map.put (word, map.getOrDefault (word, 0) + 1);
        }
        for (Map.Entry<String, Integer> entry : map.entrySet()) {
            System.out.println (entry.getKey() + " -> " + entry.getValue());
        }
    }
}
```

(vi) Check if two linked lists are equal.

Code:

```
import java.util.*;
public class LinkedListEquality {
    public static void main (String[] args) {
        LinkedList<Integer> list1 = new LinkedList<> (Arrays.asList (1, 2, 3));
        LinkedList<Integer> list2 = new LinkedList<> (Arrays.asList (1, 2, 3));
        LinkedList<Integer> list3 = new LinkedList<> (Arrays.asList (1, 3, 2));
        System.out.println ("list1 = list2: " + list1.equals (list2));
    }
}
```

```
System.out.println ("list1 = list3: " + list1.equals (list3));
}
```

6. How does Java handle XML file using DOM and SAX Parsers? Compare both approaches with respect to memory usage, processing speed and use cases. Provide a scenario where SAX would be preferred over DOM.

XML stands for extensible Markup Language. It is a based format used to store, organize and transport data in a structured and readable way. It is used:

- i) Web services
- ii) Configuration files
- iii) Data interchange between systems etc.

Java provides two main ways to parse and handle data → i) DOM parser

ii) SAX Parser

• Use of DOM parser to handle XML data:

- i) Loads the entire XML file into memory and creates a tree structure.
- ii) The tree can be traversed in any direction.

iii) Allows random access to nodes.

• Advantages of using DOM:

- i) Easy to navigate and modify XML
- ii) Best for small to medium sized XML files.

• Use of SAX Parser to handle XML data:

- i) Reads XML sequentially using event driven callbacks.
- ii) Does not load the whole XML file into memory.
- iii) Only forward traversal is possible.

• Advantages of using SAX:

- i) Low memory usage.
- ii) Very fast for large XML files.

Comparison between DOM and SAX is given below:

DOM Parser	SAX Parser
i) It loads entire XML into memory.	i) It processes XML sequentially, don't load fully.
ii) Its processing speed is slower.	ii) Its processing speed is faster.
iii) Access data randomly.	iii) Access data sequentially.
iv) Can read and modify XML.	iv) Only can read and process XML once.
v) Used when XML files are reasonably small.	v) Used when XML files are large.

Scenario when SAX is preferred over DOM:

Scenario:

You work in a library system that receives a daily XML file named books.xml. It contains thousands of books. You only need to print the titles of books published after 2015 and don't need to modify or store the full XML.

For handling this problem SAX is best choice because:

- i) It reads line by line, triggers an event when it finds `<book>` or `<year>`
- ii) You can check the year, print the title if matches and move on.
- iii) Fast and memory efficient.

7. How does the virtual DOM in React improve performance. Compare it with the traditional DOM and explain the diffing algorithm with a simple component update example.

React's Virtual DOM (VDOM) is a key innovation that boosts performance and efficiency, especially for dynamic and interactive UIs. VDOM is a lightweight in-memory copy of the actual DOM. React keeps a virtual representation of the UI in JavaScript.

How Virtual DOM improves performance in React is given below:

1. Initial Render

2. state or Props update →

When a component's state change, React -

- Creates a new virtual DOM.
- Compares it to the previous vDOM.

3. Efficient Reconciliation:

React figures out →

- What's changed
- What needs to be added, updated or removed.

Comparison between VDOM and traditional DOM:

Feature	VDOM	Traditional DOM
Update Approach	Direct Use a virtual copy to compute minimal update	Direct DOM manipulation
Performance	Faster - because avoid unnecessary operations	Slower
Re-rendering	Automatic and optimized through diffing	Manual updates on full re-renders.
Ease of maintenance	Easier	Harder for dynamic UIs

Explanation of diffing algo with a simple component update example:

React's diffing algo is a part of its reconciliation process.

When the UI updates, React:

1. Builds a new Virtual DOM.
2. Compare it with the previous vDOM.
3. calculates the minimum set of changes needed.
4. Efficiently updates the real DOM based on the differences.

Simple component example:

Initial state

```
function Greeting() {
```

```
  return <h1>Hello, Afifa! </h1>;
```

```
  type: 'h1',
  props: {
```

```
    children: 'Hello Afifa!'
```

After state change

```
function Greeting() {
```

```
  return <h1>Hello, Tahsin! </h1>;
```

```
  type: 'h1',
  props: {
```

```
    children: 'Hello, Tahsin!'
```

```
  }
```


What is Event Delegation in Javascript? and how does it optimize performance? Explain with an example of a click event on dynamically added elements.

Event delegation is a technique where a single event listener is added to a parent element and it handles events for its child element using event bubbling.

Event delegation reduces/optimizes the performance by reducing the number of event listeners needed in a web page. Instead of adding individual listeners to each child element, a single listener is attached to a parent element. This takes advantages of event bubbling, where events on child elements propagate up to their parents. When an event occurs on a child, parent's listener handles it, effectively managing event for all descendants.

Example of click event on dynamically added elements:

Context:

You have an empty `<ul>` list. Users can click a button to add new `<li>` items. When any `<li>` is clicked, you want to show an alert with the item's text. If you use event delegation, you add just one listener

to the `<ul>` but if you use no event delegation then you'll need to attach a click listener to every new `<li>`.

9. Explain how Java regular expressions can be used for input validation. Write a regex pattern to validate an email address and describe how it works using the Pattern and Matcher classes.

Java Regular expressions are now a powerful toolkit for input validation, allowing you to define patterns that strings must match to be considered valid. They're particularly useful for validating formats like email address, phone numbers etc.

Java provides regex functionality through the `java.util.regex` package, primarily using two classes:

Pattern: Represents a compiled regular expression.

Matcher: Performs matching operations on a character sequence using a pattern.

Regex pattern for a basic email:

```
^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,6}$
```


Here,

- ^ → start of the string
- [a-zA-Z0-9._%+-]+ → Local part
- @ → must include @ symbol
- [a-zA-Z0-9.-]+ → Domain name
- \\. → Dot (.)
- [a-zA-Z]{2,6} → Top level domain (com, org)
- \$ → End of the string

How the regex pattern works using Pattern and Matcher Class is given below:

Pattern Class:

- Pattern.compile(regex) - compiles regex into a pattern object.
- Pattern.matches(regex, input) → static method for one time matching

Matcher Class:

- matcher.matches() → Tests if entire string matches the pattern.
- matcher.find() → Finds the next occurrence of the pattern
- matcher.reset() → Resets the matcher to reuse it.

10. What are custom annotations in Java and how can they be used to influence program behaviour at runtime using reflection? Design a simple custom annotation and show how it can be processed with annotated elements.

Custom annotations are user defined metadata that can be attached to code elements. When combined with reflection they let your program →

- i) Detect annotations at runtime
- ii) Read their values
- iii) Change behaviour dynamically

Step by step process of how custom annotation works is given below →

- i) Define a custom annotation using @interface.
- ii) Use @Retention to make it accessible at runtime.
- iii) Attach the annotation to classes, methods or fields.
- iv) Use Java Reflection API to read and process the annotations.
- v) Apply logic/behaviour based on the annotation value.

How custom annotation influence program behaviour at runtime using Reflection is given below:

1. Define a custom annotation

```
import java.lang.annotation.*;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface RUNNow {
    int times() default 1;
}
```

Use the Annotation in a class

```
public class MyTask {
    @RUNNow(times = 3)
    public void printHello() {
        System.out.println("Hello");
    }
    public void notAnnotated() {
        System.out.println("This won't be called");
    }
}
```

2. Process it with reflection at Runtime

```
import java.lang.reflect.Method;
public class AnnotationRunner {
    public static void main(String[] args) throws Exception {
        MyTask task = new MyTask();
        for (Method method : task.getClass().getDeclaredMethods()) {
```

```
if (method.isAnnotationPresent(RUNNow.class)) {
    RUNNow annotation = method.getAnnotation(RUNNow.class);
    int times = annotation.times();
    for (int i = 0; i < times; i++) {
        method.invoke(task);
    }
}
```

Output:

```
Hello
Hello
Hello
```

11. Discuss the Singleton design pattern in java. What problem what does it solve and how does it ensure only one instance of a class is created? Extend your answer to explain how thread safety can be achieved in a Singleton implementation

The Singleton design Pattern is one of the most well known creational patterns that ensures a class has only one instance throughout the application lifecycle while providing global access to that instance.

The Singleton Pattern addresses several key issues:

- i) Preventing multiple instances
- ii) Only one object is needed to coordinate actions across the system.
- iii) Resource management.

The pattern uses several mechanisms to guarantee that only one instance is created. They are -

- i) Private construction
- ii) Static Factory Method
- iii) Handling reflection attacks.
- iv) Serialization safety
- v)

To achieve thread safety in a Singleton implementation there are some methods:

i) Synchronized method

Code:

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
    public static synchronized Singleton getInstance()  
    {  
        if (instance == null) {
```

```
            instance = new Singleton();
```

```
        }  
        return instance;  
    }  
}
```

2. Bill Pugh Singleton

Code:

```
public class Singleton {  
    private Singleton() {}  
    private static class Holder {  
        private static final Singleton INSTANCE = new  
        Singleton();  
    }  
    public static Singleton getInstance() {  
        return Holder.INSTANCE;  
    }  
}
```


12. Describe how JDBC manages communication between a Java application and a relational database. Outline the steps involved in executing a SELECT query and fetching results. Include error handling with try-catch and finally blocks.

JDBC is an API that allows Java applications with to interact with relational databases like MySQL, PostgreSQL and Oracle etc.

How JDBC manages communication:

JDBC acts as a bridge between Java and the database using:

- i) DriverManager - loads the JDBC driver
- ii) Connection - establishes the session
- iii) Statement - sends SQL commands
- iv) ResultSet - holds the data returned from queries.

Here's a concise outline of the steps to execute a SELECT query using JDBC with error handling:

1. Load the JDBC Driver
2. Establish a connection
3. Prepare SQL query.

4. Set parameters

5. Execute the query

6. Process the ResultSet

7. Handle Exceptions

8. Close resource in finally block.

Example of catch and finally block:

```
try {  
    //set parameters, execute  
}  
catch (SQLException e) {  
    //handle sql errors  
}  
finally {  
    //close resultSet, statement  
}
```


13. How do servlets and JSPs work together in a web application following the MVC architecture? Provide a brief use case showing the servlet as a controller, JSP as a view and a Java class as the model.

In a Java web application using MVC architecture, servlets, JSPs and Java classes each play distinct roles to cleanly separate logic, presentation and data.

How they work together is given below:

1. Client sends an HTTP request
2. Servlet receives the request
3. Extracts data from the request
4. Forward result to JSP for display
5. Model performs data processing
6. JSP (view) displays the result.

Use case:

Java class as model

UserValidator.java

```
public class UserValidator {
    public static boolean isValid(String username, String password) {
        return username.equals("admin") && password.equals("1234");
    }
}
```

View using JSP

result.jsp

```
<%
Boolean isValid = (Boolean) request.getAttribute("isValid");
if (isValid != null && isValid) {
    %>
    <h2> Welcome, admin! </h2>
    <% } else {
    <h2> Login failed. Try again. </h2>
    <% } %>
```

Controller using servlet

LoginServlet.java

```
@webServlet("/login")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        String user = request.getParameter("username");
        String pass = request.getParameter("password");
        boolean isValid = UserValidator.isValid(user, pass);
        request.setAttribute("isValid", isValid);
        RequestDispatcher rd = request.getRequestDispatcher(
            "result.jsp");
        rd.forward(request, response);
    }
}
```


14. Explain the life cycle of Java Servlet. What are the roles of `init()`, `service()`, `destroy()` methods? Discuss how servlets handle concurrent requests and how thread safety issues may arise.

The servlet life cycle defines the stages a servlet goes through from its creation to destruction in a servlet container. The `javax.servlet` package provides interfaces for handling these stages.

Roles of methods:

`init()`

- Called once after the servlet is instantiated.
- Used to initialize resources.
- The `ServletConfig` object provides access to initialization parameters.

`service()`

- Called every time the servlet receives a request.
- It dispatches to `doGet()`, `doPost()` etc, depending on request method.

`destroy()`

- Called once before the servlet is taken out of service.
- Clean up resources like closing DB connections.

How Servlet handles concurrent Requests is given below and also why thread safety matters -

Servlet Request Handling.

- The Servlet container (Tomcat) creates one instance of each servlet class.
- For every incoming request, the container spawns a new thread and invokes the servlet's `service` method.
- This means many threads can be executing the servlet at the same time.

Thread safety concerns

Since multiple threads share the same servlet instance, any shared or mutable instance variables can be accessed by multiple threads concurrently. This can lead to -

- Race conditions
- Inconsistent results
- Unexpected behaviour.

15. A single instance of a servlet handles multiple requests using threads. What problem can occur if shared resources are accessed by multiple threads? Illustrate your answer with an example and suggest a solution using synchronization.

When multiple threads access shared resources then the given problems arise:

In a servlet one instance serves many requests simultaneously. The servlet container spawns a new thread for each request.

If instance variables are read or modified by these threads without coordination, problems arise - such as:

- i) Race conditions → threads interfere with each other unpredictably.
- ii) Data inconsistency → variables can have unexpected values.
- iii) Incorrect results → one user's data may leak into another user's response.

An example problem using race condition and its solution using synchronization is given below:

Example:

```
private int counter = 0;  
counter++;  
response.getWriter().println("Request number: " + counter);
```

In this problem, 2 threads run nearly at the same time.

Solution:

To prevent this problem, we can use synchronization. Synchronization means access to the shared variable so only one thread at a time can execute that critical section.

```
synchronization(this){  
    counter++;  
    response.getWriter().println("Request number: " + counter);  
}
```

Here the synchronization -

- Locks the servlet instance before entering the block.
- Ensures only one thread updates counter at a time.

16. Describe how the MVC pattern separates concerns in a Java web application. Explain the advantages of this structure in terms of maintainability and scalability using a student registration system as an example.

The MVC pattern is a widely used design pattern in Java web applications, especially with Spring MVC, Struts and JSF etc frameworks. It separates concerns by dividing the application into three main components each responsible for a specific aspect of the application.

1) Model → Business logic and data

- i) Responsibility: Manages the application data, rules & logic
- ii) Examples: Java classes that represent data, service classes and DAO.
- iii) Concern separated: Business logic is isolated from UI and request handling.

2) View:

- i) Responsibility: Displays data to the user and handles user interface.
- ii) Example: JSP, Thymeleaf etc.
- iii) Concern separated: UI rendering is kept separate from data and control logic.

3) Controller:

- i) Responsibility: Acts as an intermediary between View and Model.
- ii) Example: Spring @Controller classes.
- iii) Concern separated: Input processing and routing are separated from data logic and presentation.

Using student registration system, the advantages of MVC pattern in terms of maintainability and scalability is given below:

Maintainability:

- If there is a bug in data storage, you only check the Model.
- UI designers can work on JSP without touching business logic.

Scalability:

- You can plug in different views using the same Controller and Model.
- New features (e.g. 'view student list') can be added by just introducing new components without breaking existing ones.

17. In a Java EE application, how does a servlet controller manage the flow between the model and the view? Provide a brief example that demonstrates forwarding data from a servlet to a JSP and rendering a response.

In a Java EE application, a servlet controller manages the flow between the model and the view by:

1. Receiving requests from the client
2. Calling model classes to process data or fetch from the database.
3. setting attributes on the request scope.

Flow overview:

Client → Servlet → Model → JSP(view) → Client
(controller)

Example: Student Info

1. Model class

```
public class Student {  
    private String name;  
    private int age;  
    public Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
}
```

```
public String getName(); { return name; }  
public int getAge() {  
    return age; }  
}
```

2. JSP(view) :

```
<%@ page imports="your.package.student" %>  
<%  
    Student student = (Student) request.getAttribute("student  
Data");  
%>  
<html>  
    <head> <title>Student Info </title> </head>  
    <body>  
        <h2>Student details </h2>  
        <p> Name: <% = student.getName() %> </p>  
        <p> Age: <% = student.getAge() %> </p>  
    </body>  
</html>
```

3. Controller(Servlet) :

```
@webServlet("/student")  
public class StudentServlet extends HttpServlet {  
    protected void doGet(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
        Student student = new Student("Afifa", 22);  
        request.setAttribute("studentData", student);  
    }  
}
```



```

RequestDispatcher dispatcher = request.getRequestDispatcher(
    "student.jsp");
dispatcher.forward(request, response); } }

```

is. compare and contrast cookies, URL rewriting and HttpSession as methods for session tracking in Servlets. Discuss their advantages, limitations and ideal use cases.

The comparison table is given below:

Feature	Cookies	URL rewriting	HttpSession
Mechanism	Stores session id on data in browser cookies.	Appends session id to the URL as a parameter.	Stores data on server side with session ID in client.
Storage Location	Client	Client	Server
Persistence	Can persist beyond browser session.	Only active during session.	Exists for session duration.
Automatic Handling	Yes	No	Yes
Security	Medium	Low	High
Data Capacity	Limited	Limited	High

Advantages, limitations and ideal use cases of each method is given below:

Cookies

- Small piece of data stored on the client side.
- Sent with each HTTP request.
- Used to track user info.

Advantages:

- Persistent across sessions.
- Easy to implement.

Limitations:

- Limited in size.
- Security risks.

Ideal use case:

- Remembering user preferences.

URL rewriting

- Session ID is manually added to every URL as parameter.
- Useful when cookies are disabled.

Advantages

- No reliance on browser cookies.
- Simple for all small apps.

Session Timeout and Invalidation

• Session Timeout

1. A session will expire automatically after a period of inactivity.
2. This is configured in web.xml.
3. After the timeout, the session object is destroyed, and the session ID becomes valid.

Invalidation

1. You can explicitly invalidate a session using `request.getSession().invalidate();`

Secure Handling of Session

1. Use `HttpOnly` and `Secure` cookies
2. Use `HTTPS`
3. Regenerate Session on Login
4. Validate Session Attributes
5. Short Timeout for sensitive Data.

20. Explain how Spring MVC handles an HTTP request from a browser. Describe the role of the `@Controller`, `@RequestMapping` and Model objects in separating business logic from presentation. Provide a brief flow example of a login form submission.

In Spring MVC, handling an HTTP request involves a well structured flow that follows the Model-View-Controller pattern. This architecture separates business logic, request handling, and presentation making the application modular, maintainable and testable.

Spring MVC Request Handling Flow

When a browser sends a request -

1. The `DispatcherServlet` receives the request
2. It looks for a controller method mapped to the request URL.
3. The matched `@Controller` method processes the request.
4. It uses a model object to add attributes for the view.

5. The method returns a viewname and the DispatcherServlet renders it.

The role of ^{annotations} business logic in separating business logic is given below:

1. @Controller →

- Keeps the business logic in services
- Connects request routing to backend processing.

2. @RequestMapping →

- Directs with controller method responds to a given URL.
- Allows clean Routing separate from both business logic and UI rendering.

3. Model Object →

- Avoids embedding data processing logic in views.
- Keeps UI and backend logic loosely coupled.

Example of login form submission -

@controller

```
public class LoginController {
```

```
    @PostMapping("/login")
```

```
    public String handleLogin(
```

```
        @RequestParam String username;
```

```
        @RequestParam String password;
```

```
        Model model) {  
    if ("admin".equals(username) && "pass".equals(password))  
        model.addAttribute("user", username);  
    return "loginSuccess";  
    }  
    else {  
        model.addAttribute("error", "Invalid credentials");  
        return "loginForm";  
    }  
}
```

21. Spring MVC uses the DispatcherServlet as a front controller. Describe its role in the request processing workflow. How does it interact with view resolvers and handler mappings?

DispatcherServlet Interaction Diagram:

Browser Request



DispatcherServlet



HandlerMapping → Controller Method



Model → View Name



ViewResolver

↓
Render view with model

↓
Browser receives HTML response

Role of DispatcherServlet in Request Processing:

1. Receives request:

- All requests mapped to spring MVC are first received by DispatcherServlet.

2. Delegates to Handler Mappings:

- The servlet consults HandlerMapping beans to determine which controller method should handle the request.

3. Invokes the controller

- The identified controller method is invoked with any parameters.

4. Processes Return Value

- The controller method returns a logical view with data.

5. Resolves the View:

- DispatcherServlet uses a ViewResolver to convert the logical view name to an actual view.

6. Renders the view.

23. What is a ResultSet in JDBC and how it is used to retrieve data from a MySQL database? Briefly explain the use of next(), getString() and getInt() methods with an example.

In JDBC, a ResultSet is an object that holds the data returned from executing a select query on a relational data such as: MySQL. It represents a table of data. Each row in the result-set can be accessed one at a time, using a cursor that move forward.

How it retrieves data is given below:

(.rs.next() moves the first row.)

- Establish a database connection
- create a statement
- Execute the query
- Iterate through the ResultSet
- Retrieve column values.
- Process the retrieved data.
- Close Resources.

Use of common methods with example:

- `next()`: Moves the cursor to the next row. Returns false when there are no more rows.
- `getString()`: Retrieves the value of the specified column as a string.
- `getInt()`: Retrieves the value of the specified column as an int.

Example:

```
String query = "select id, name, marks from students";
ResultSet rs = stmt.executeQuery(query);
while (rs.next()) {
    int id = rs.getInt("id");
    String name = rs.getString("name");
    int marks = rs.getInt("marks");
    System.out.println(id + " | " + name + " | " + marks);
}
```

24. How does JPA manage the mapping between Java objects and relational tables? Explain with an example using `@Entity`, `@Id` and `@GeneratedValue` annotations. Discuss the advantage of using JPA over raw JDBC.

JPA manage the mapping between Java objects and relational databases tables through a process called Object-Relational Mapping.

Here's how JPA handle the mapping:

Entity definition

- Java classes are marked as JPA entities using the `@Entity` annotation. This indicates that the class represents a table in the database.

Field to Column Mapping

- By default, JPA maps fields in an entity class to columns in the corresponding database table with the same name.

Relationship mapping

JPA provides annotations to manage relationships between entities, which correspond to relationships between tables in the database.

Example, with @Entity, @Id and @GeneratedValue annotation:

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
```

@Entity

```
public class Student {
```

@Id

```
    private int id;
```

```
    private String name;
```

```
    public int getId() {
```

```
        return id; }
```

```
    public void setId(int id) {
```

```
        this.id = id; }
```

```
    public String getName() {
```

```
        return name; }
```

```
    public void setName(String name) {
```

```
        this.name = name; }
```

```
}
```

The advantages of using JPA over raw JDBC for database interaction Java applications -

i) Object-Relational Mapping

ii) Reduced Boilerplate Code

iii) Database Independence

iv) Improved maintainability and Readability

v) Enhanced Productivity

24. Design a simple CRUD applications using Spring and MySQL to manage student records. Describe how each operation would be implemented, using a repository interface.

Project overview:

i) Add a student (create)

ii) Get all or single student (read)

iii) Update a student's data (update)

iv) Delete a student (delete)

① Database Table

create database studentdb;

use studentdb;

② Entity class - Student.java

```
import jakarta.persistence;
```

@Entity

```
public class Student {
```



```

@Id
@GeneratedValue
private long id;
private String name;
-----
}

```

3. Repository Interface

```

import org.springframework.data.jpa.repository.
    JpaRepository;
public interface StudentRepository extends JpaRe-
    pository<Student, Long> {
    -----
}

```

This interface gives built in methods like

- save()
- findById(), findAll()
- deleteById()

4. Controller class

```

import org.springframework.web.bind.annotation;
import java.util.List;
@RestController
@RequestMapping("/students")

```

```

public class StudentController {
    private final StudentService service;
    public StudentController(StudentService service) {
        this.service = service;
    }
    // create
    @PostMapping
    public Student addStudent(@RequestBody Student student) {
        return service.createStudent(student);
    }
    // add
    // update
    // delete
}

```


27

27. How does Spring Boot simplify the development of RESTful services? Describe how to implement a REST controller using `@RestController`, `@GetMapping` and `@PostMapping`, including JSON data handling.

Spring Boot greatly simplifies the development of RESTful services using:

i) Auto configuration

- Automatically configures web servers and JSON converters with minimal setup.

ii) Embedded server:

- No need to deploy WARs to an external server. Just run your app like a Java application.

iii) Spring web starter

- Includes all required dependencies for building REST API's.

iv) Reduced Boilerplate

- Annotations like `@RestController`, `@GetMapping`, `@PostMapping` etc. simplify request handling.

Implementation of REST controller using annotations and JSON data handling:

1) `@RestController` Annotation:

- Marks the class as a REST API controller.

- It combines `@Controller` and `@ResponseBody`.

2) `@GetMapping`

- `@GetMapping` is used to map HTTP GET requests.
- Typically used to retrieve data from server.

3) `@PostMapping`

- `@PostMapping` maps HTTP post requests.
- Used to send data to server.
- Conversion here is handled using `@RequestBody`.

4) JSON data handling

- The JSON is sent by the client.
- Spring converts this JSON into a Java object using Jackson.
- This is done by making the method parameter with `@RequestBody`.

22

22. How does PreparedStatement improve performance and security over statement in JDBC? Write a short example to insert a record into a MySQL table using PreparedStatement.

How PreparedStatement improves Performance & security over statement in JDBC is given below:

Security:

- PreparedStatement automatically escapes user input making it safe from SQL injection attacks.
- With Statement, user input is directly concatenated into the SQL string.

Performance:

- PreparedStatement allows the database to pre-compile the SQL statement and reuse it with different parameters.
- This reduces parsing time and improves efficiency.

Example

Inserting a Record Using PreparedStatement

```
import java.sql.*;  
public class InsertExample {
```

```
    public static void main (String[] args) {  
        String url = "jdbc:mysql://localhost:3306/studentdb";  
        String username = "maisha";  
        String pass = "root-1236";  
        String insertSQL = "insert into students (name, age)  
            values (?, ?)";  
        try (Connection conn = DriverManager.getConnection(  
            url, username, password);  
            pstmt = conn.prepareStatement(insertSQL)) {  
            pstmt.setString(1, "Afifa");  
            pstmt.setInt(2, 22);  
            int rowsInserted = pstmt.executeUpdate();  
            if (rowsInserted > 0)  
                System.out.println("Successful insertion");  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}
```


(25)

Describe the difference between the Entity Managers `persist()`, `merge()` and `remove()` operations. When would you use each method in a typical database transaction scenario.

Difference table is given below:

Method	Purpose	Return value	When to use
<code>persist()</code>	Makes a new entity managed and scheduled it for insertion	void	When inserting a new record
<code>merge()</code>	copies the state of a detached entity into a managed entity	Managed entity	When updating an entity
<code>remove()</code>	Deletes a managed entity from the database	void	When deleting a record.

Detailed explanation and used Scenarios:

1. `persist()` (object entity):

- Used for saving new entities to the db.
- Adds the object to the persistence context and inserts it into the database at commit.
- Entity must be transient.

2. `merge()`

- used for updating detached entities
- Return a new managed copy of the entity with its state copied from the input.
- Entity can be detached / transient.

3. `remove()`

- Used for deleting an entity from database.
- Marks the entity for deletion, it is removed at commit.
- Entity must be managed.

28. Compare @RestController and @Controller in Spring Boot. In a RESTful API for a library system, describe how you would structure the endpoints for books using REST principles.

Comparison between @RestController and @Controller is given below:

Feature	@Controller	@RestController
Purpose	Used for rendering views	Used for RESTful APIs
Returns	Returns View Name	Returns Response Body
Requires @ResponseBody	Yes	No
Use Case	Traditional MVC apps	Web services

Restful API structure for a library system:

HTTP Method	Endpoint	Description
GET	/books	Retrieve all books
GET	/books/{id}	Retrieve a book by Id.

POST	/books	Add a new book
PUT	/books/{id}	Update an existing book
DELETE	/books/{id}	Delete a book by Id

Benefits given by this structure:

- Intuitive: Easy to understand and use by clients.
- Consistent: Follows uniform interface principle.
- Cacheable: GET responses can be cached.

③

You are building a multi module spring boot application. Explain how you would structure the project using Maven or Gradle and manage inter module dependencies effectively.

In a multi-module Spring Boot Application, the goal is to separate concerns into modular, reusable, components, improving maintainability, testability and accessibility:

Project structure (Maven or Gradle):

springboot-multimodule/

├ pom.xml

├ common/

├ pom.xml

├ repository/

├ pom.xml

├ service

├ pom.xml

├ api

├ pom.xml

So here

api is responsible for exposing REST endpoints and it depends on service

• service is responsible for business logic and depends on repository.

• repository handles DB access

• common is responsible for DTOs, enums, utilities and depends on nothing.

Example of inter module dependencies:

<dependencies>

<dependency>

<groupId> com.example </groupId>

<artifactId> common </artifactId>

<version> 1.0.0 </version>

</dependency>

<dependency>

<groupId> com.example </groupId>

<artifactId> repository </artifactId>

<version> 1.0.0 </version>

</dependency>

</dependencies>

30. Compare Maven and Gradle as build tools in Spring Boot projects. Discuss their syntax, performance and dependency handling with an example of a build.gradle configuration for a simple REST API.

Comparison table of Maven and Gradle:

Feature	Maven	Gradle
Syntax	XML	Groovy/Kotlin DSL
Readability	Verbose	Concise, more legible
Performance	Slower	Faster
Custom Logic	Harder to script	Easily scriptable
Build output	Typically .jar/.war	Same, but more flexible
Tooling support	Excellent	Growing
Community support	Mature, very widely adopted	Increasing rapidly

Example for a simple Spring Boot REST API

Using gradle:

plugins {

id 'java'

id 'org.springframework.boot' version '3.2.0'

id 'io.spring.dependency-management' version '1.1'

group = 'com.example'

version = '0.0.1-SNAPSHOT'

sourceCompatibility = '17'

repositories {

mavenCentral()

}

dependencies {

implementation 'org.springframework.boot:spring-

boot-starter-web'

implementation 'com.fasterxml.jackson.core

jackson-databind'

}

test {

useJUnitPlatform()

}

29

21. How does maven manage dependencies and build lifecycle in a spring boot project. Explain the structure of a typical pom.xml and how the spring boot starter dependencies simplify development.

How Maven manages dependencies and build lifecycle is given below:

Dependencies:

- Maven uses the pom.xml file to declare external libraries.
- It downloads required JAR files from Maven central.
- Developers don't need to manually download or manage JARs.

Build lifecycle:

Maven has a predefined build lifecycle consisting of phases.

- validate - checks if the project is correct.
- compile - compiles the source code.
- test - runs unit tests.
- package - packages code.

Typical pom.xml structure

```
<?xml version="1.0" encoding="UTF-8" ?>
<modelVersion>4.0.0</modelVersion>
<groupId>com.example</groupId>
<artifactId>student-app</artifactId>
<version>1.0.0</version>
<packaging>jar</packaging>
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version>
  <relativePath/>
</parent>
```

How spring boot starter dependencies simplify development is given below:

- Spring MVC
- Jackson
- Embedded Tomcat
- Logging

1. Demonstrate how a child class can access a protected member of its parent class within the same package. Explain with example what happens when the child class is in a different package.

= In Java a protected member is a variable, method or constructor. And these members can be accessed by-

- i) Within same package any class can access protected members.
- ii) In different package only subclass can access protected members and only through inheritance.

Let's look at both the scenarios →

Case 1: child class in same package

File: Parent.java

```
package mypackage;
public class Parent {
    protected String message = "Hello from Parent";
}
```

File: Child.java

```
package mypackage;
public class Child extends Parent {
    public void showMessage() {
        System.out.println(message);
    }
}
```

```
public static void main(String[] args) {
```

```
    Child c = new Child();
    c.showMessage();
}
```

Output:

Hello from Parent

Case 2: child class in a different package

File: Parent.java

```
package mypackage;
public class Parent {
    protected String msg = "Hello I am Maisha";
}
```

File: Child.java [in another package]

```
package childpackage;
import mypackage.Parent;
public class Child extends Parent {
    public void message() {
        System.out.println(msg);
    }

    public static void main(String[] args) {
        Child c = new Child();
        c.message();
    }
}
```

Output:

Hello I am Maisha